Pico Handheld Game Console

A dual-core, DMA-driven handheld built on the Raspberry Pi Pico 2 (RP2350)

Project Report

Names:	Aditi Dure, Eppala Mohana
Roll number:	EE23BTECH11016, EE23BTECH11018
Course:	Embedded Systems
Repository:	`https://github.com/aditidure/handheld_embd`
Date:	November 2026

Abstract

This report describes the design and implementation of a USB-powered handheld game console based on the Raspberry Pi Pico 2 (RP2350 microcontroller). The console drives a 320×240 ST7789 colour display over SPI, reads an analog joystick and five push buttons, and produces sound through a PWM signal, a PAM class-D amplifier and a speaker. It boots into a game-select menu with two games, *Space Shooter* (physics-based) and *Snake*.

All graphics are drawn into a RAM framebuffer and sent to the display by DMA. The two CPU cores form a pipeline: core 0 runs the game and draws the next frame into one buffer while core 1 sends the previous frame from a second buffer. The display then runs at 51.4 FPS (frames per second), a limit set by the SPI link, while core 0 needs only 3.0 ms of each 19.5 ms frame. Buttons are sampled by a 1 kHz timer interrupt with a 5 ms debounce filter and press latching, so presses are neither missed nor doubled. Best scores are stored in on-board flash with wear levelling and CRC32 checks. The console also has non-blocking sound effects, a pause screen, and a sleep mode that turns the display off, lowers the CPU clock from 150 to 48 MHz and waits for a button press.

Contents

1 Introduction **2**

1.1 Objective . . . 2

1.2 Requirement compliance . . . 2

1.3 Feature summary . . . 3

2 System Overview **3**

2.1 Bill of materials . . . 4

3 Hardware **4**

3.1 Raspberry Pi Pico 2 and the RP2350 . . . 4

3.1.1 Clock tree and CPU frequency . . . 4

3.1.2 Memory hierarchy . . . 5

3.2 Display: ST7789 over SPI . . . 6

3.2.1 Signals . . . 6

3.2.2 Initialisation and commands . . . 6

3.2.3 SPI speed: maximum vs. operating . . . 7

3.3 Input: analog joystick . . . 7

3.3.1 ADC and multiplexer . . . 8

3.3.2 Calibration and normalisation . . . 8

3.4 Input: push buttons . . . 9

3.5 Audio: PWM, PAM amplifier and speaker . . . 9

3.6 Power distribution . . . 10

3.7 Pin assignment . . . 10

4 Software Architecture **11**

4.1 Modules . . . 11

4.2 Execution model . . . 11

4.3 Boot sequence . . . 12

5 Rendering Pipeline **12**
5.1 Framebuffer and pixel format . . . 12
5.2 Sprites and drawing primitives . . . 12
5.3 DMA transfer . . . 13
5.4 Dual-core double buffering . . . 13
5.5 FPS measurement . . . 14

6 Input Handling **14**
6.1 Problems solved . . . 14
6.2 Design: interrupt producer, frame consumer . . . 15
6.3 Timing figures . . . 15

7 Audio Engine **16**
7.1 Tone generation with PWM . . . 16
7.2 Non-blocking sound effects . . . 16

8 Persistent Scores in Flash **17**
8.1 Flash constraints . . . 17
8.2 Record format . . . 17
8.3 Algorithms . . . 17

9 Power Management and Sleep **18**

10 Games and State Machines **19**
10.1 Menu . . . 19
10.2 Space Shooter . . . 19
10.3 Snake . . . 21
10.4 Display task (core 1) . . . 21

11 Measurements and Results **21**
11.1 CPU frequency . . . 21
11.2 SPI speed . . . 21
11.3 Frame rate . . . 22
11.4 CPU profile . . . 22
11.5 Polling and update frequencies . . . 23
11.6 Debounce and joystick timing . . . 23
11.7 Memory usage . . . 24
11.8 Asset sizes . . . 24

12 Testing and Validation **24**

13 Limitations and Future Work **25**

14 Conclusion **25**

A Building and Flashing **25**

B Controls **26**

C Glossary **26**

1 Introduction

1.1 Objective

The aim of the project is to design and build a self-contained, USB-powered handheld that runs a playable game with sound, persistent scores and good responsiveness. The brief sets the following requirements.

- **Baseline hardware:** a Raspberry Pi Pico board, an SPI display, push buttons, USB (or battery) power, and an easy-to-hold enclosure.
- **Baseline software:** sprites with framebuffer-based rendering (with measured FPS), score display and storage, clean input handling (no missed or bounced presses), and use of both CPU cores.
- **Additions:** DMA for framebuffers, sound, sleep on close, physics-based game mechanics, and multi-console play.
- **Deliverables:** working hardware and its schematic, commented source code on GitHub, a report with measurements (FPS, memory usage, energy usage), and a video.

1.2 Requirement compliance

Table 1 compares the brief with what was built, and states openly which items were not done.

Table 1: Requirements of the brief and their status.

Requirement	Status	How it is met / notes
Raspberry Pi Pico board	**Yes**	Pico 2 (RP2350), dual Cortex-M33 at 150 MHz
SPI display	**Yes**	ST7789 colour LCD, 320 × 240, SPI0 (Section 3.2)
Buttons	**Yes**	5 buttons (A, B, C, D and the joystick click) plus a 2-axis analog joystick
USB powered	**Yes**	Fully powered from USB; USB also carries the serial log
Easy-to-hold enclosure	**No**	Not built; the electronics are used on a bench
Framebuffer rendering with sprites	**Yes**	Full-screen RGB565 framebuffer; ship, asteroids, snake and icons are drawn as procedural sprites from primitives and bitmaps (Section 5)
FPS measured and reported	**Yes**	Measured every second, shown on screen and logged over USB (Section 11.3)
Score display	**Yes**	Live score and best score in every game HUD and on the menu cards
Score storage	**Yes**	Flash, with wear levelling and CRC32 (Section 8)
Clean input handling	**Yes**	1 kHz interrupt sampling, 5 ms debounce, press latching (Section 6)
Dual-core usage	**Yes**	Core 0 runs the game, core 1 drives the display (Section 5.4)
DMA for framebuffers	**Yes**	Each 153 600-byte frame is sent by DMA
Sound	**Yes**	PWM → PAM class-D amplifier → speaker, non-blocking (Section 7)

Requirement	Status	How it is met / notes
Sleep on close	Yes	Hold-to-sleep and 60 s idle timeout (no lid) (Section 9)
Physics-based mechanics	Yes	Ship acceleration, friction and speed limit (Section 10.2)
Multi-console play	No	Not implemented
Schematic	No	Not made
Source on GitHub	Yes	Commented, builds with no compiler warnings
Measurements	Partial	Clocks, SPI, FPS, timing and memory measured; energy not measured
Video	To add	

1.3 Feature summary

- Game-select menu with two games, each showing its saved best score.
- **Space Shooter**: a physics-driven ship shooting falling asteroids in three directions, collectable shield charges, a timed shield, particle explosions, score pop-ups and a level banner every 100 points.
- **Snake**: a 32×21 grid that speeds up with every apple.
- Pause screen in both games, with every game timer frozen and restored.
- On-screen FPS counter (toggle) and a once-per-second timing log over USB.
- Sleep mode by holding the joystick click, or after 60 s idle.

2 System Overview

Figure 1 shows the complete system. The Pico 2 sits in the middle and connects to every other part through one of its built-in peripherals.

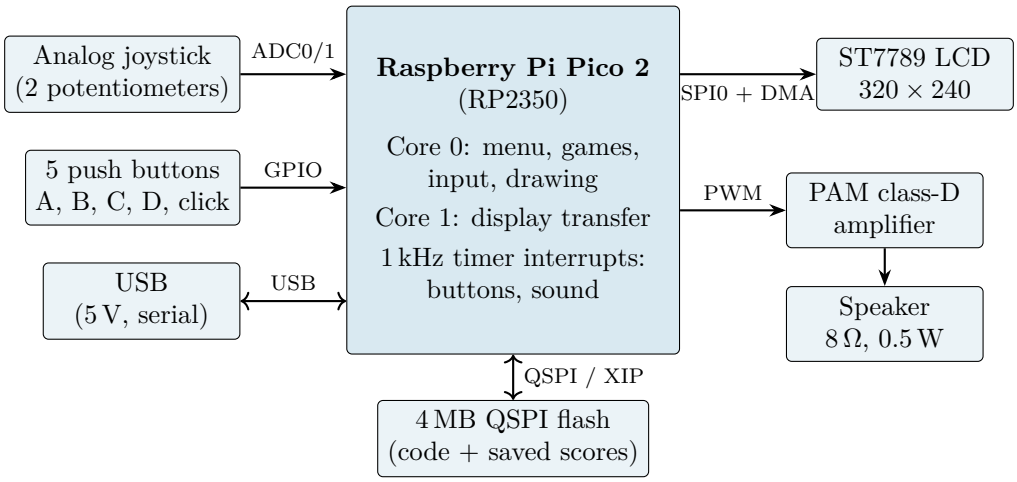


Figure 1: System block diagram.

2.1 Bill of materials

Table 2: Bill of materials.

Item	Qty	Notes
Raspberry Pi Pico 2 (RP2350)	1	520 KB SRAM, 4 MB flash
ST7789 SPI colour LCD module	1	Panel 240 × 320, used in landscape
Joystick shield	1	2-axis analog stick with push switch, buttons A–D
PAM8302 class-D amplifier	1	Mono, powered from 3.3 V, shutdown pin
Speaker	1	8 Ω, 0.5 W
Breadboard and jumper wires	–	
USB cable	1	Power and data

3 Hardware

3.1 Raspberry Pi Pico 2 and the RP2350

The Pico 2 is a small board built around the RP2350 microcontroller. Table 3 lists the parts of the chip this project relies on.

Table 3: RP2350 features and how the project uses them.

Feature	Detail and use
CPU	Two Arm Cortex-M33 cores at 150 MHz, each with a single-precision floating-point unit (FPU). Core 0 runs the game; core 1 runs the display task.
SRAM	520 KB: eight striped 64 KB banks plus two 4 KB “scratch” banks used for the core stacks. Holds both framebuffers.
Flash	4 MB external QSPI flash. Code runs directly from it (execute-in-place, XIP) through a 16 KB cache. The last 4 KB stores best scores.
Clocks	One 12 MHz crystal; phase-locked loops (PLLs) generate 150 MHz (system) and 48 MHz (USB, ADC).
SIO	Single-cycle I/O block private to each core: fast GPIO, inter-core FIFOs, event signals.
Peripherals used	SPI0, 1 DMA channel, ADC (2 inputs), PWM slice 7, timer alarms, USB device, flash interface.

3.1.1 Clock tree and CPU frequency

Every clock is derived from a 12 MHz quartz crystal. The system PLL multiplies it up to a 1500 MHz internal oscillator (the VCO) and two post-dividers bring it down to 150 MHz:

$$f_{\text{sys}} = \frac{12\,\text{MHz} \times 125}{5 \times 2} = 150\,\text{MHz}.$$

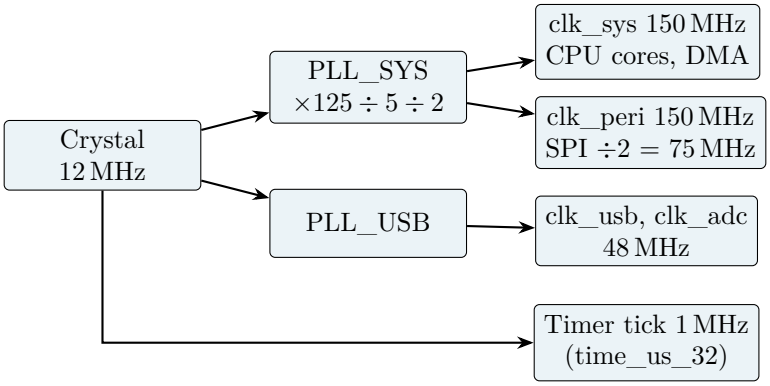


Figure 2: Clock tree as used by the project.

Table 4: Clock frequencies.

Clock	Normal	Sleep	Used for
clk_sys	150 MHz	48 MHz	Both CPU cores, DMA, memory bus
clk_peri	150 MHz	–	SPI (divided by 2 to 75 MHz)
clk_adc	48 MHz	48 MHz	ADC (96 cycles per conversion)
clk_usb	48 MHz	48 MHz	USB serial
Timer tick	1 MHz	1 MHz	`time_us_32()`, all periodic interrupts

The `benchmark_test` program checks every clock with the RP2350's hardware frequency counter, which counts each clock against the 12 MHz crystal. All measured values match the configured ones: clk_sys = 150 000 kHz, clk_peri = 150 000 kHz, clk_adc = clk_usb = 48 000 kHz, and clk_ref = 12 001 kHz (the counter's ±1 kHz resolution, 0.008 %).

3.1.2 Memory hierarchy

Table 5: Memory hierarchy from fastest to slowest.

Level	Size	Access time	Holds in this project
CPU registers	16 + 32 FPU	0 extra cycles	Values being computed
SIO (per core)	–	1 cycle (6.7 ns)	GPIO reads/writes (CS, DC, buttons)
SRAM	520 KB	1 cycle if no bank conflict	Framebuffers, game state, stacks
XIP cache	16 KB	≈1 cycle on a hit	Recently executed code
External flash	4 MB	tens of cycles on a cache miss	Program, constant data, saved scores
Display GRAM	1 frame	19.5 ms per full frame	The image being shown

Key point. The Cortex-M33 cores in the RP2350 have no data cache, so both cores and the DMA always see the same RAM contents. Correct sharing therefore only needs `volatile` variables and memory barriers, not cache maintenance.

SRAM banking. The eight main banks are interleaved word by word, so consecutive 4-byte words fall in different banks. A framebuffer is therefore spread across all banks, and core 0 writing the back buffer rarely collides with the DMA reading the front buffer in the same cycle. The core stacks live in the two small scratch banks, so stack traffic never competes with frame data.

3.2 Display: ST7789 over SPI

The display is a TFT LCD module driven by an ST7789 controller. The controller holds the image in its own memory (GRAM) and refreshes the panel from it, so the picture stays on screen between transfers. The Pico writes to it over SPI; the display never sends data back.

3.2.1 Signals

Table 6: Display signals.

Signal	Pin	Function
SCK	GP18	SPI clock. One pulse per bit; driven by the SPI hardware.
MOSI (SDA)	GP19	SPI data, most significant bit first.
CS	GP17	Chip select, active low. Frames each transaction; driven by software.
DC	GP16	Data/command: 0 = command byte, 1 = parameter or pixel data.
RST	GP20	Hardware reset, active low.
VCC, BL	3V3	Logic supply and backlight LEDs. BL is tied to 3.3 V, so the backlight is always on.

SPI runs in mode 0: the clock idles low, the Pico changes the data bit while the clock is low, and the display samples it on the rising edge. Eight clock pulses carry one byte.

3.2.2 Initialisation and commands

After a hardware reset pulse (RST low for 10 ms, then 120 ms to restart), the driver sends the commands in Table 7.

Table 7: ST7789 commands used.

Code	Name	Purpose
`0x01`	SWRESET	Software reset
`0x11`	SLPOUT	Leave sleep mode (wait 120 ms afterwards)
`0x3A` + `0x55`	COLMOD	16 bits per pixel (RGB565)
`0x36` + `0x60`	MADCTL	Memory access order: landscape (MX + MV bits)
`0x20`	INVOFF	Colour inversion off
`0x13`	NORON	Normal display mode
`0x29`	DISPON	Display on
`0x2A`/`0x2B`	CASET/RASET	Set the column/row window
`0x2C`	RAMWR	Pixel data follows
`0x28`/`0x10`	DISPOFF/SLPIN	Display off / sleep (used by sleep mode)

Rotation. The panel is physically 240 × 320 (portrait). MADCTL value `0x60` sets bit MV, which swaps rows and columns, and bit MX, which mirrors horizontally to undo the reflection the swap introduces. The display then behaves as 320 × 240, and the drawing code simply uses `SCREEN_WIDTH` = 320 and `SCREEN_HEIGHT` = 240. If the image appears upside down, `0xA0` gives the other landscape orientation.

3.2.3 SPI speed: maximum vs. operating

The SPI hardware divides the 150 MHz peripheral clock by an even prescaler (at least 2) and a post-divider:

$$f_{\mathrm{SCK}} = \frac{f_{\mathrm{peri}}}{\mathrm{prescale} \times \mathrm{postdiv}}, \qquad \mathrm{prescale} \in \{2, 4, \ldots, 254\},\ \mathrm{postdiv} \in \{1, \ldots, 256\}.$$

The code requests 80 MHz. The SDK chooses the fastest rate not above the request, $150/(2 \times 1) = 75$ MHz.

Table 8: SPI speeds.

Quantity	Value
Requested (`SPI_BAUDRATE`)	80 MHz
RP2350 maximum ($f_{\mathrm{peri}}/2$)	75 MHz
Operating clock (chosen by the SDK)	75 MHz
ST7789 datasheet limit (16 ns min. write cycle)	62.5 MHz
Measured effective throughput	63.0 Mbit/s
Actual rate reported by `spi_get_baudrate()`	75 MHz (confirmed)

The panel runs about 20 % above its datasheet limit. This works reliably on this module and gives the highest frame rate available.

Frame transfer time. One frame is 320 × 240 × 2 = 153 600 bytes = 1 228 800 bits.

$$t_{\mathrm{ideal}} = \frac{1\,228\,800}{75 \times 10^6} = 16.4\,\mathrm{ms}, \qquad t_{\mathrm{measured}} = 19.5\,\mathrm{ms}.$$

The difference comes from short gaps between bytes (the DMA feeds the SPI one byte at a time), the window-setting commands sent before each frame, and waiting for the last bits to leave the SPI hardware. The effective throughput is 1 228 800/19.5 ms ≈ 63 Mbit/s, about 84 % of the clock rate. This transfer time is the hard upper limit on frame rate: 1/19.5 ms ≈ 51 FPS.

3.3 Input: analog joystick

The joystick contains two 10 kΩ potentiometers wired as voltage dividers between 3.3 V and ground. Moving the stick moves each wiper, giving a voltage from 0 to 3.3 V; at rest each sits near the middle.

- X axis → GP26 (ADC input 0); Y axis → GP27 (ADC input 1).
- The joystick must be powered from 3.3 V: the ADC pins must not exceed it.
- The push switch (joystick click) is a digital button on GP8 (Section 3.4).

3.3.1 ADC and multiplexer

The RP2350 has one 12-bit successive-approximation ADC shared between its inputs through a multiplexer (mux). A conversion takes 96 cycles of the 48 MHz ADC clock:

$$t_{\text{conv}} = \frac{96}{48\,\text{MHz}} = 2\,\mu\text{s} \quad \Rightarrow \quad 500\,\text{k samples/s}.$$

Reading X then Y means switching the mux (`adc_select_input()`, a single register write) and converting each time.

Table 9: ADC timing.

Quantity	Expected	Measured
Resolution	12 bits (0–4095), 0.8 mV per step	–
Conversion (`adc_read`)	2.0 µs (hardware)	2.50 µs
Full `joystick_read_x()`	≈2–3 µs	3.33 µs

Interpretation.

- **Conversion:** the hardware takes 2.0 μs; the measured 2.50 μs adds about 0.5 μs of software: starting the conversion, polling the "ready" flag and reading the result over the peripheral bus, where each register access costs several cycles. Polling one sample at a time therefore gives 399 k samples/s; the full 500 k samples/s would need the ADC's free-running mode with DMA.
- **Cost per frame:** `joystick_read_x()` (select, convert, floating-point normalisation) takes 3.33 μs; both axes take about 6.7 μs, 0.03 % of a 19.5 ms frame.

3.3.2 Calibration and normalisation

The measured raw values of this joystick are listed below. The X centre is noticeably off the ideal 2048 and its maximum never reaches 4095.

	Min	Centre	Max
X	0	1769	3747
Y	6	2073	4095

Each side of the centre is scaled separately (piecewise-linear), so both extremes reach exactly ± 1:

$$r = \begin{cases} \dfrac{v - c}{c - v_{\min}} & v < c \\[2ex] \dfrac{v - c}{v_{\max} - c} & v \geq c \end{cases} \qquad \text{then clamp to } [-1, 1], \quad |r| < 0.2 \Rightarrow r = 0.$$

The final step is a 20 % *dead zone*: small readings caused by electrical noise, or by the spring not returning exactly to centre, become exactly zero so the ship does not drift. Example: X raw 885 gives $(885 - 1769)/1769 = -0.50$.

How the games use it. Space Shooter uses the value proportionally (half a push gives half the acceleration). Snake converts it to four directions with a 0.6 threshold on the dominant axis. The menu uses hysteresis: a step when $|x| > 0.6$, re-armed only after $|x| < 0.3$, so one push moves exactly one card. Screen y grows downwards, so the games use $-y$.

3.4 Input: push buttons

Each button connects a GPIO pin to ground. The pin's internal pull-up resistor (about 50 kΩ) holds it at logic 1 when released; pressing pulls it to 0. The buttons are therefore *active-low*, and no external resistors are needed.

Button	Pin	Main use
A	GP2	Fire up; hold 3 s on the menu to reset scores
B	GP3	Fire right; next game on the menu
C	GP4	Shield; FPS overlay toggle on the menu
D	GP5	Fire left; previous game; back to menu
Joystick click	GP8	Start, pause/resume; hold 2 s on the menu to sleep

3.5 Audio: PWM, PAM amplifier and speaker

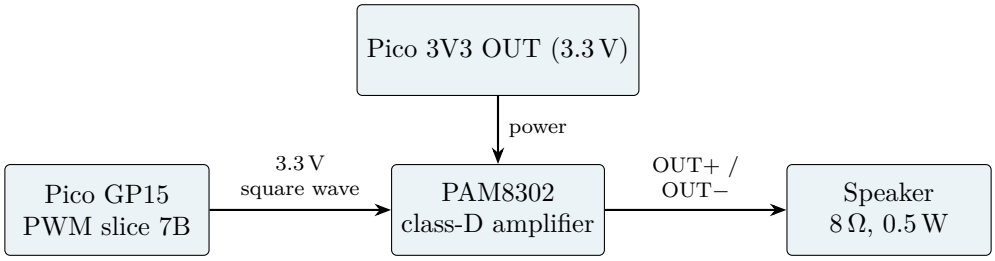


Figure 3: Audio signal chain.

Why an amplifier. A speaker has an impedance of a few ohms. At 3.3 V an 8 Ω speaker would try to draw about 0.4 A, far above the few milliamps a GPIO pin can supply. The amplifier takes the weak logic-level signal and drives the speaker with power taken from the Pico's 3.3 V supply.

Class-D operation. A class-D amplifier switches its output transistors fully on or off at several hundred kilohertz, itself a form of PWM. The speaker coil cannot follow such fast switching and averages it, reproducing only the audio. Because the transistors are always fully on or fully off, very little power is lost as heat (about 85–90 % efficiency), so no heatsink is needed.

The PAM8302 module. The PAM8302 is a mono class-D amplifier that runs from 2.0–5.5 V; here it is powered from the Pico's 3V3 OUT pin. Its input is differential (A+/A−): GP15 drives A+ and A− goes to ground. The output power is limited by the supply voltage. With a bridge-tied output the peak voltage across the speaker is about V_{DD}, so

$$P_{\max} \approx \frac{V_{DD}^2}{2R} = \frac{(3.3)^2}{2 \times 8} \approx 0.68\,\text{W}$$

in theory, and roughly 0.5 W in practice after losses. This matches the 0.5 W speaker, so it cannot be badly overdriven. (At 5 V the same module could deliver about 1.5 W into 8 Ω.) Loud sounds can draw current peaks of around 0.2 A from the 3.3 V rail, which also feeds the display, so a decoupling capacitor (e.g. 100 µF) across the amplifier's supply pins helps keep the rail steady. The module also has a shutdown (SD) pin, which could switch the amplifier off during sleep (Section 9).

Bridge-tied output. The PAM output drives both speaker wires in opposite directions, doubling the voltage swing. The speaker's negative wire must therefore go to OUT−, never to ground.

Ground. The Pico and the amplifier share one ground; otherwise the input signal has no common reference.

3.6 Power distribution

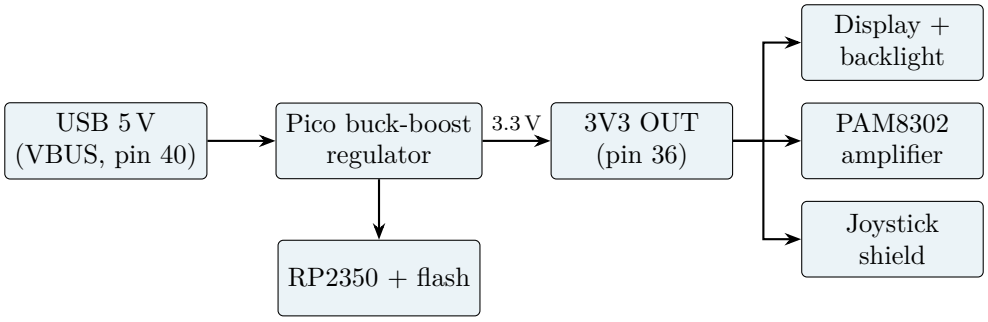


Figure 4: Power distribution. All grounds are connected together.

The Pico's regulator produces a steady 3.3 V for the RP2350, its flash, and the 3V3 OUT pin; inside the chip a further regulator makes the core supply of about 1.1 V. Every external module (display, amplifier and joystick) is powered from 3V3 OUT, so the whole handheld runs on one 3.3 V rail. The Pico documentation recommends drawing no more than about 300 mA from 3V3 OUT; the display and backlight take a few tens of milliamps and the amplifier draws its peaks only while a sound plays, so this budget is respected.

3.7 Pin assignment

Table 10: Complete pin assignment.

Pin	GPIO	Connected to	Peripheral	Notes
4	GP2	Button A	GPIO in	Pull-up, active low
5	GP3	Button B	GPIO in	Pull-up, active low
6	GP4	Button C	GPIO in	Pull-up, active low
7	GP5	Button D	GPIO in	Pull-up, active low
11	GP8	Joystick click	GPIO in	Pull-up, active low
20	GP15	Amplifier input	PWM 7B	Amp input ground to Pico GND
21	GP16	Display DC	GPIO out	
22	GP17	Display CS	GPIO out	
24	GP18	Display SCK	SPI0 SCK	75 MHz
25	GP19	Display MOSI/SDA	SPI0 TX	
26	GP20	Display RST	GPIO out	
31	GP26	Joystick X	ADC0	Max 3.3 V
32	GP27	Joystick Y	ADC1	Max 3.3 V
36	3V3	VCC of all modules	Power	Display, BL, joystick, amplifier
38 etc.	GND	All grounds	Ground	Common ground

4 Software Architecture

4.1 Modules

The firmware is written in C on the Raspberry Pi Pico SDK 2.3.1. Each hardware function lives in its own module with a small header. Table 11 lists every file; the sizes are the code and constant data each module contributes to flash, and the RAM it reserves.

Table 11: Source modules (sizes from the compiled object files).

File	Responsibility	Flash (B)	RAM (B)
`handheld.c`	Entry point: hardware init, load scores, start menu	52	0
`menu.c`	Game-select menu, FPS toggle, score reset, hold-to-sleep	1 997	4
`space_shooter.c`	Space Shooter game	11 676	3 383
`snake.c`	Snake game	2 784	1 384
`display/display.c`	ST7789 driver, framebuffers, DMA, core 1 task, drawing, font, FPS statistics	5 077	307 279
`input/buttons.c`	1 kHz sampling, debounce, press latching	640	48
`input/joystick.c`	ADC reading, calibration, dead zone	552	0
`sound/sound.c`	Non-blocking PWM sound effects	820	50
`save/save.c`	Best scores in flash	704	284
`power/power.c`	Sleep and wake	570	4
`ui/ui.c`	Shared drawing helpers, colours, pause box	1 311	0
Application total		**26 183**	**312 436**
Pico SDK + C library	USB stack, printf, maths, start-up, multicore, flash	39 013	5 292
Firmware total		**65 196**	**317 728**

The test programs in `test/` (`display_test`, `buttons_test`, `joystick_test`, `sound_test` and `benchmark_test`) reuse the same modules.

4.2 Execution model

Three kinds of execution context run at the same time:

Table 12: Execution contexts.

Context	Where	Work
Main loop	Core 0	Menu or game: read input snapshot, update logic, draw the frame, call `display_show()`
Display task	Core 1	Wait for a finished frame, set the window, run the DMA transfer, signal completion
Button interrupt	Core 0, 1 kHz	Sample 5 pins, debounce, latch presses
Sound interrupt	Core 0, 1 kHz	Advance the current sound effect, update the PWM frequency

4.3 Boot sequence

1. The boot ROM checks BOOTSEL, finds the program's image definition in flash and starts it on core 0. Core 1 waits in the boot ROM.
2. The C runtime copies initialised data to RAM, zeroes `.bss` (including the framebuffers), sets the clocks to 150 MHz and calls `main()`.
3. `main()` starts USB serial, then initialises the display (which launches core 1), buttons, joystick and sound.
4. Saved best scores are loaded from flash and the random generator is seeded from the timer.
5. `menu_run()` starts and never returns.

5 Rendering Pipeline

5.1 Framebuffer and pixel format

Every frame is drawn into a RAM array of 320×240 pixels, then sent to the display in one transfer. The screen never shows a half-drawn picture, so there is no flicker.

Each pixel is 16 bits in RGB565 format: 5 bits red, 6 bits green, 5 bits blue. Green gets the extra bit because the eye is most sensitive to it. A 24-bit colour is converted with

$$\mathrm{RGB565}(r, g, b) = ((r \,\&\, \texttt{0xF8}) \ll 8) \mid ((g \,\&\, \texttt{0xFC}) \ll 3) \mid (b \gg 3).$$

The RP2350 stores 16-bit values low byte first, but the ST7789 expects the high byte first, so each pixel is byte-swapped when it is written. The DMA can then copy the buffer to SPI unchanged.

5.2 Sprites and drawing primitives

The game objects are *procedural sprites*: they are drawn from geometric primitives at run time rather than copied from stored images. Small fixed graphics (the 5×7 font and the heart icon) are stored as bitmaps.

Table 13: Drawing primitives.

Primitive	Algorithm
Line	Bresenham's algorithm: integer-only stepping along the major axis
Filled rectangle	Clipped to the screen, then a row-by-row fill
Circle outline	Midpoint circle algorithm, drawing 8 symmetric points per step
Filled circle	One horizontal span per row, half-width $\sqrt{r^2 - d_y^2}$
Filled triangle	Vertices sorted by y; each scanline filled between the two interpolated edges
Text	5×7 bitmap font (39 glyphs), scaled by drawing each set bit as a square

Examples: the ship is three filled triangles (two wings and the hull), a filled circle (cockpit) and a flickering triangular flame. An asteroid is a filled circle plus an 8-vertex jagged outline rotated by $(\cos\theta, \sin\theta)$ each frame, plus two crater circles. The cost of each primitive is measured by `benchmark_test` (Section 11.4).

5.3 DMA transfer

One DMA channel is configured to copy 153 600 bytes from the framebuffer to the SPI data register:

- transfer size 8 bits; read address increments, write address fixed;
- paced by the SPI transmit request signal (DREQ), so the DMA writes only when the SPI has room;
- before the DMA starts, the CPU sends CASET, RASET and RAMWR with DC toggled, then holds DC = 1 and CS = 0 for the whole burst.

The DMA moves the data with no CPU involvement. Some code must still wait for it to finish before raising CS and reusing the buffer, and that is the job given to core 1.

5.4 Dual-core double buffering

With one framebuffer, core 0 must wait 19.5 ms for every transfer, because it cannot draw into memory that is being sent. With two framebuffers, core 0 draws the next frame while core 1 sends the previous one (Figure 5).

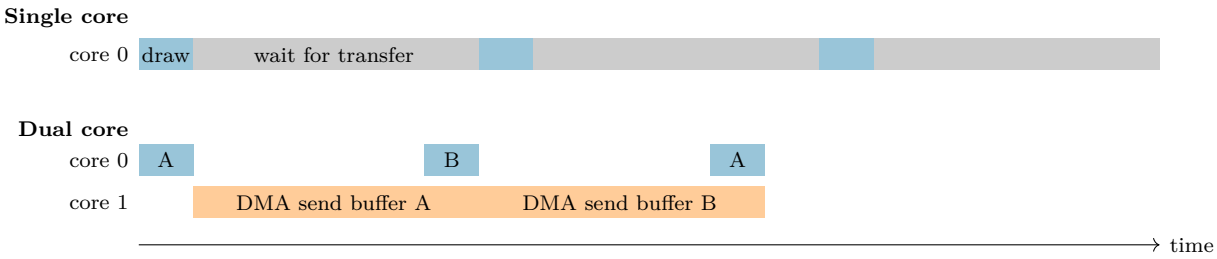


Figure 5: Frame pipeline. Core 0 draws in about 3 ms; a transfer takes 19.5 ms. In dual-core mode the transfer overlaps the drawing of the next frame.

Hand-off protocol. The two cores share three variables: `transfer_busy`, `transfer_index` and `core1_transfer_us`. Core 0, in `display_show()`:

```
while (transfer_busy) __wfe(); // wait until core 1 finished the last frame
__dmb();
transfer_index = back_index; // hand over the frame just drawn
__dmb();
transfer_busy = true;
__sev(); // wake core 1
back_index ^= 1; // draw the next frame in the other buffer
framebuffer = framebuffers[back_index];
```

Core 1 runs forever:

```
flash_safe_execute_core_init(); // allow core 0 to pause us during flash writes
while (true) {
    while (!transfer_busy) __wfe(); // sleep until there is a frame
    __dmb();
    transfer_frame(framebuffers[transfer_index]); // window + DMA + wait
    __dmb();
    transfer_busy = false;
    __sev(); // wake core 0 if it is waiting
}
```

- `volatile` forces each read of a shared variable to come from memory instead of a stale register copy.
- `__dmb()` (data memory barrier) makes sure the buffer index is written before `transfer_busy` becomes visible to the other core.
- `__wfe()` (wait for event) puts a core to sleep until the other core executes `__sev()` (send event). Each core has a one-bit event flag that remembers a pending event, so a wake-up sent just before the other core goes to sleep is never lost.

Why this split. The transfer is the largest cost (19.5 ms against 3 ms of drawing) and is completely independent of game logic once a frame is finished, so moving it to core 1 gives the largest gain. The two cores share only three small variables, so no locks are needed in the game code, and the games themselves did not change: they still just call `display_show()`. Splitting the drawing itself between the cores would gain little, since drawing takes only 3 ms. Splitting game logic from drawing would require locking all game state.

Build option. The CMake option `HANDHELD_DUAL_CORE=OFF` builds a single-core version (one buffer, blocking transfer) from the same source, for the comparison in Section 11.3.

5.5 FPS measurement

`display_show()` timestamps every frame with the 1 MHz hardware timer. Over each one-second window it computes:

- FPS = frames / elapsed time;
- average and worst frame time;
- average DMA transfer time (measured on core 1);
- average time core 0 spent *waiting* for the display;
- core 0 *render* time = frame time − wait time (logic + drawing).

The results are printed over USB once per second and can be shown on screen (press C on the menu).

6 Input Handling

6.1 Problems solved

1. **Contact bounce.** A mechanical contact bounces open and closed for 1–5 ms when pressed or released, so one press can look like several.
2. **Missed presses.** A game reading the pins once per frame (every 19.5 ms) can miss a short tap that falls between two reads.
3. **Inconsistent reads.** Different parts of one frame reading the pin at different moments could disagree.

6.2 Design: interrupt producer, frame consumer

A repeating timer interrupt runs every 1 ms (polling frequency 1 kHz) on core 0 and performs a counter-based debounce:

```
bool raw_down = (gpio_get(pin) == 0); // active-low
if (raw_down == stable_down[i]) { // agrees: nothing to do
    change_count[i] = 0;
} else if (++change_count[i] >= BUTTON_DEBOUNCE_MS) { // 5 ms in a row
    stable_down[i] = raw_down; // accept the new state
    change_count[i] = 0;
    if (raw_down) pending_presses[i]++; // latch the press
}
```

A change is accepted only after the pin has read the new value for 5 ms in a row; any bounce back resets the count. Every accepted press increments a counter, so it is remembered until the game reads it.

Table 14: Debounce example: a press with one bounce.

ms	Raw pin	Count	Stable state
0	0 (pressed)	1	released
1	1 (bounce)	0 (reset)	released
2	0	1	released
3–5	0	2–4	released
6	0	5	**pressed**, press counter +1

Once per frame the game calls `buttons_poll()`, which copies the press counters into a per-frame snapshot and clears them. Interrupts are disabled for these few microseconds (a *critical section*), so a press arriving between the copy and the clear cannot be lost. The game then asks:

- `button_just_pressed(b)`: true for exactly one frame per press (pause, shield, menu steps, Snake turns);
- `button_down(b)`: true while held (continuous firing, hold timers).

`buttons_clear()` discards unread presses when switching screens, so the press that leaves a game cannot also act in the menu.

6.3 Timing figures

Table 15: Input timing.

Quantity	Value
Button polling frequency	1 kHz (every 1 ms)
Debounce time	5 ms
Press recognised after the contact closes	5–6 ms
Shortest press accepted	5 ms
Presses remembered between frames	up to 255 per button
Joystick polling	once per frame per axis (≈51 Hz)

End-to-end input latency (estimate). From pressing a button to seeing the result on screen:

$$\underbrace{5\text{–}6}_{\text{debounce}} + \underbrace{0\text{–}19.5}_{\text{until next poll}} + \underbrace{3}_{\text{logic + draw}} + \underbrace{19.5}_{\text{transfer}} \approx 28\text{–}48\,\text{ms}.$$

This is below the roughly 50 ms at which delay becomes noticeable to a player. The display's own refresh adds up to one more panel refresh period on top. The latency can be measured by filming the screen and a button press with a high-speed phone camera.

7 Audio Engine

7.1 Tone generation with PWM

A PWM slice is a 16-bit counter that counts from 0 to TOP and wraps; the output is high while the counter is below LEVEL. With clock divider DIV:

$$f = \frac{f_{\text{sys}}}{\text{DIV} \times (\text{TOP} + 1)}, \qquad \text{duty} = \frac{\text{LEVEL}}{\text{TOP} + 1}.$$

TOP cannot exceed 65 535, so without a divider the lowest note would be 150 MHz/65 536 ≈ 2.3 kHz. The code chooses the smallest divider that keeps TOP within 16 bits:

$$\text{DIV} = \left\lfloor \frac{f_{\text{sys}}}{65\,536\, f} \right\rfloor + 1, \qquad \text{TOP} = \frac{f_{\text{sys}}}{\text{DIV} \cdot f} - 1, \qquad \text{LEVEL} = \text{TOP}/2.$$

Example, A4 = 440 Hz: DIV = 6, TOP = 56 817, LEVEL = 28 408, giving $150 \times 10^6/(6 \times 56\,818) = 440.0$ Hz. The usable range is about 20 Hz to 8 kHz. The duty cycle is fixed at 50 %, which gives the strongest fundamental tone.

Note frequencies follow equal temperament, $f = 440 \times 2^{(n-69)/12}$ (MIDI note n), and are stored as constants in `sound/notes.h`.

7.2 Non-blocking sound effects

The original code played tones with `sleep_ms()`, freezing the game for every beep. The engine now runs in a 1 kHz timer interrupt. A sound effect is a list of segments, each sliding its pitch linearly from a start to an end frequency over a number of milliseconds:

$$f(t) = f_{\text{start}} + (f_{\text{end}} - f_{\text{start}})\,\frac{t}{T}.$$

Explosion-like sounds add pseudo-random jitter of ±50 % every millisecond, using a 16-bit linear-feedback shift register (LFSR), a very fast pseudo-random generator. Each effect has a priority, so a frequent laser shot can never cut off an important sound such as being hit.

Table 16: Sound effects.

Effect	Segments (Hz, ms)	Character	Priority
Shoot	1800→700, 45	falling "pew"	1
Explosion	900→250, 160, noise	crunch	2
Shield blocks rock	2000, 25; 1400, 50	metallic "ting"	2
Orb pickup	880, 45; 1319, 45; 1760, 70	rising chime	3
Shield on	500→1800, 160	rising sweep	3

Effect	Segments (Hz, ms)	Character	Priority
Hit	700→250, 120 noise; 300→150, 160 noise	heavy crash	4
Snake eats	1047, 40; 1568, 60	two-note blip	2
Snake crash	800→200, 250, noise	crunch	4
Menu move	1300, 25	blip	1
Menu select	600→1800, 120	sweep	5

8 Persistent Scores in Flash

8.1 Flash constraints

- **Erase** works on whole 4 KB sectors and sets every bit to 1. **Program** works on 256 byte pages and can only change bits from 1 to 0, so data cannot be overwritten in place.
- Each sector survives about 10^5 erase cycles.
- While flash is being erased or programmed it cannot be read, so no code can execute from it. Both interrupts and core 1 must be paused.

8.2 Record format

The last 4 KB sector of flash (offset `0x3FF000`) is divided into 16 pages. Each save writes one 24-byte record into the next empty page.

Table 17: Save record (24 bytes, padded to a 256-byte page with `0xFF`).

Offset	Field	Size	Purpose
0	magic	4 B	`0x48485343` (“HHSC”): marks our data
4	version	4 B	Record format version (1)
8	sequence	4 B	Increases with each save; the highest is newest
12	scores[0]	4 B	Space Shooter best
16	scores[1]	4 B	Snake best
20	checksum	4 B	CRC32 of bytes 0–19

8.3 Algorithms

Loading. At start-up all 16 pages are scanned; a page counts as valid only if its magic number, version and CRC32 all match, and the valid record with the highest sequence number wins. A record half-written when power was lost fails the CRC and is ignored, so the previous good record is used.

Saving (wear levelling). A score is written only when it beats the stored best. The record goes into the next page; the sector is erased only when all 16 pages are used, or the next page is not blank. This gives 16 times fewer erases than rewriting one fixed location:

$$\text{lifetime} \approx 10^5 \text{ erases} \times 16 \text{ saves per erase} = 1.6 \times 10^6 \text{ saves.}$$

Safe writes. The write runs inside `flash_safe_execute()`, which disables interrupts and parks core 1 (registered with `flash_safe_execute_core_init()`) in RAM while flash is busy. The SDK flash routines themselves run from RAM and flush the XIP cache afterwards.

Verification. The module was tested on a PC against a simulated flash chip: scores survived a simulated reboot; 41 saves caused only 3 sector erases; a corrupted newest record correctly fell back to the previous one; and nothing outside the save sector was modified.

Reset. Holding A for 3 s on the menu resets all best scores.

9 Power Management and Sleep

The handheld has no lid, so "sleep on close" is triggered from software:

- holding the joystick click for 2 s on the menu, or
- 60 s without any input on a calm screen (menu, title, pause, game over; never during play).

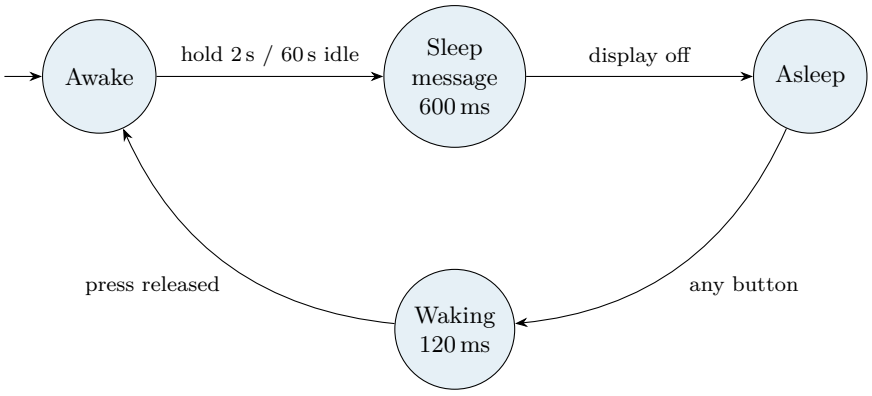


Figure 6: Sleep state machine.

Table 18: What happens on sleep and wake.

Step	Action
1	Stop all sound
2	Show "SLEEPING" for 600 ms
3	Wait for core 1 to finish any transfer, then send DISPOFF and SLPIN
4	Wait until the triggering button is released
5	Lower clk_sys from 150 to 48 MHz
6	Core 0 sleeps in `__wfi()` (wait for interrupt), checking the buttons each time the 1 ms interrupt wakes it; core 1 is already idle in `__wfe()`
7	On a press: restore 150 MHz, send SLPOUT, wait 120 ms, send DISPON
8	Wait for the release, discard the press, reset the FPS statistics, resume

Limitation. The backlight is wired straight to 3.3 V, so it stays lit during sleep and cannot be dimmed. Connecting BL to a GPIO would allow it to be switched off, which is likely the largest single saving available. The amplifier also draws a small idle current; if the module has a shutdown pin, it could be turned off during sleep the same way.

10 Games and State Machines

Every screen is implemented as a finite state machine: a variable holds the current state, and inputs or timers cause transitions.

10.1 Menu

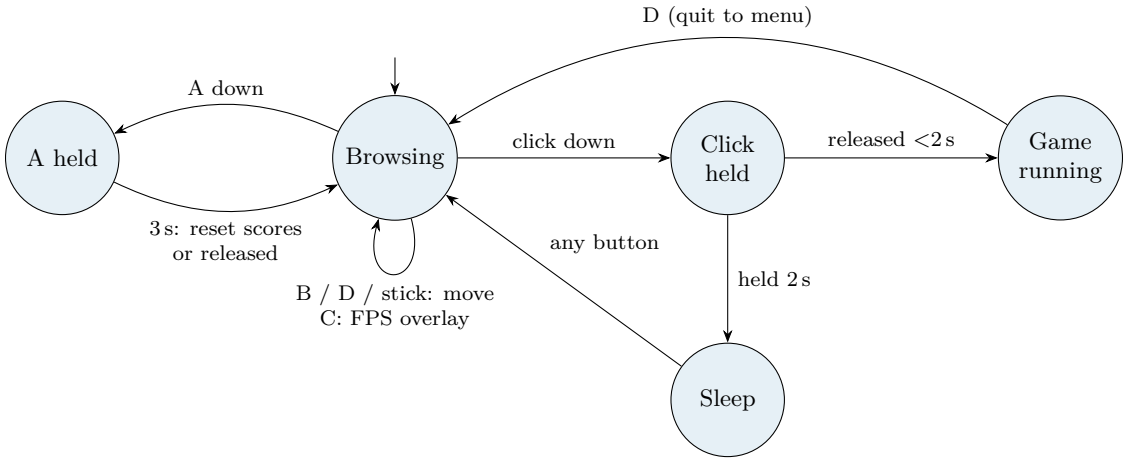


Figure 7: Menu state machine.

Games are listed in a table of {name, icon function, run function, best-score function}; adding a game means adding one entry. Holding the click starts the game only on release, so that a long hold can mean sleep instead.

10.2 Space Shooter

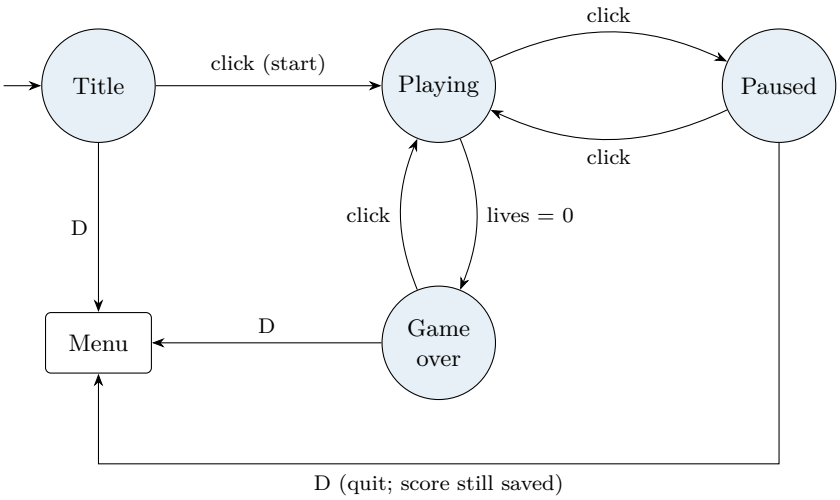


Figure 8: Space Shooter state machine.

Physics. With joystick input $u \in [-1, 1]$, acceleration a, friction coefficient k and frame time Δt, each axis is updated as

$$v \leftarrow v + a\,u\,\Delta t, \qquad v \leftarrow v\,(1 - k\,\Delta t), \qquad |v| \le v_{\max}, \qquad x \leftarrow x + v\,\Delta t.$$

At full stick the speed would settle near $a/k = 550/2.5 = 220\,\text{px/s}$, so the 180 px/s limit is reached, which keeps the ship controllable. Δt is the measured time since the last frame (capped at 50 ms), so movement speed does not depend on frame rate.

Collisions. Objects are treated as circles; two touch when $d_x^2 + d_y^2 < (r_1 + r_2)^2$. Comparing squared distances avoids a square root.

Difficulty. The asteroid spawn interval falls with score:

$$T_{\text{spawn}} = \max\left(300,\ 900 - 2\min(\text{score}, 300)\right)\ \text{ms}.$$

Pause. On resume, every stored timestamp (fire cooldowns, spawn timers, shield end time, invulnerability, orb timer, banners) is shifted forward by the pause length, so nothing expires or fires all at once.

Table 19: Space Shooter parameters.

Parameter	**Value**
Ship acceleration a	$550\,\text{px/s}^2$
Friction k	$2.5\,\text{s}^{-1}$
Maximum speed	180 px/s
Bullet speed	300 px/s
Fire cooldown per direction	180 ms (≈5.6 shots/s)
Directions	A up, B right, D left
Asteroids (max on screen)	10; radius 7–13 px; speed 65–129 px/s
Bullets (max)	12
Spawn interval	900 ms falling to 300 ms
Points per asteroid	10
Lives	3; 900 ms invulnerability after a hit
Shield orb	every 5 s; pickup radius 18 px; up to 3 charges
Shield duration	3 s (button C)
Level banner	every 100 points
Particles	up to 64; life 0.35–0.7 s
Starfield	48 stars in 3 parallax layers (12, 30, 65 px/s)

10.3 Snake

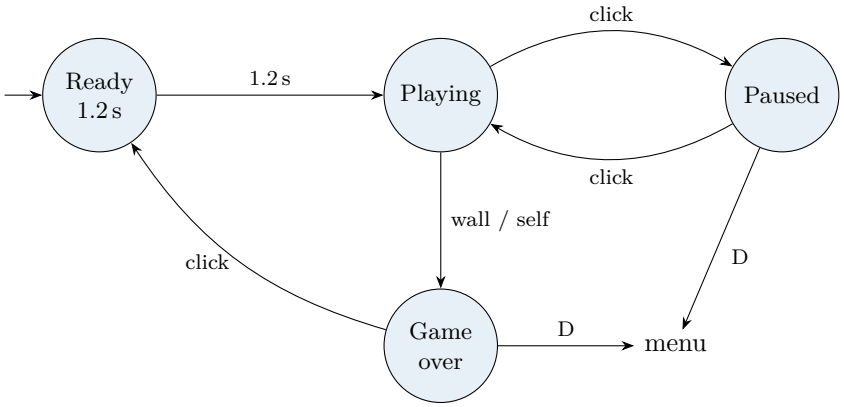


Figure 9: Snake state machine.

Table 20: Snake parameters.

Parameter	Value
Board	32×21 cells of 10 px (below a 30 px HUD)
Move interval	150 ms, −3 ms per apple, minimum 70 ms
Points per apple	10
Maximum length	672 cells
Turn rule	checked against the last *executed* move, so two quick taps cannot reverse the snake into itself

10.4 Display task (core 1)

Core 1 has two states, *Idle* (asleep in WFE) and *Transferring* (DMA running), switched by `transfer_busy`.

11 Measurements and Results

All numbers in this section were measured on the device, using the USB serial log (one line per second) and the `benchmark_test` program.

11.1 CPU frequency

See Table 4: 150 MHz during operation, 48 MHz in sleep.

11.2 SPI speed

See Table 8: 75 MHz operating clock (the RP2350 maximum), against an 80 MHz request and a 62.5 MHz panel rating; 63 Mbit/s effective.

11.3 Frame rate

Table 21: Frame rate per screen and mode.

Screen	Single core	Dual core	Dual-core worst frame
Menu	45 (predicted 44.4)	51.4	19.5 ms
Space Shooter (playing)	47	51	19.5 ms
Snake (playing)	44	51	19.5 ms

Prediction: in single-core mode the frame time is render plus transfer, so for the menu $1/(3.0 + 19.5)\,\text{ms} = 44.4$ FPS. In dual-core mode it is the larger of the two, $\max(3.0, 19.5) = 19.5$ ms, giving 51.3 FPS. The measured 45 FPS (single core) and 51.4 FPS (dual core) match both predictions. In both games the dual-core build also holds 51 FPS, against 47 (Space Shooter) and 44 (Snake) on one core. The dual-core gain, $51.4/45 = 1.14$ on the menu and up to $51/44 = 1.16$ in Snake, is 9–16 % here *because drawing is so cheap*; the benefit grows with heavier scenes, until drawing itself exceeds 19.5 ms.

11.4 CPU profile

Table 22: Where the time goes per frame (dual core).

Screen	Core 0 render	Core 0 wait	Core 0 load	Core 1 DMA
Menu	3.0 ms	16.5 ms	15 %	19.5 ms
Space Shooter	2.5 ms	17.0 ms	13 %	19.5 ms
Snake	4.6 ms	14.8 ms	24 %	19.5 ms

Core 0 load = render time / frame time. Snake costs the most to draw (4.6 ms) and Space Shooter the least (2.5 ms), but even Snake leaves core 0 idle three quarters of the time. In every case the worst frame equals the 19.5 ms transfer: drawing never makes a frame late. Core 1 spends almost the whole frame supervising the DMA transfer; most of that time is a wait, not computation.

Table 23: Cost of individual operations (from `benchmark_test`).

Operation	Time
`display_clear` (full screen)	1 283 µs (16.7 ns per pixel)
`fill_rect` (full screen)	1 301 µs
`draw_string` (19 characters)	152 µs (8 µs per character)
`draw_line` (full diagonal, 320 px)	39.9 µs (125 ns per pixel)
`fill_circle` ($r = 10$)	19.1 µs
`fill_triangle` (ship hull)	26.9 µs
`gpio_get` (incl. test loop)	120.7 ns
`joystick_read_x`	3 334 ns
Full frame, clear + show (dual core)	19 461 µs (51.4 FPS); DMA 19.5 ms; core 0 wait 18.1 ms

Interpretation. Clearing the framebuffer is the largest single drawing cost (1.28 ms, about 2.5 cycles per pixel, roughly 43 % of the menu's 3.0 ms render time); writing two pixels per 32-bit store

or clearing with DMA could make it several times faster. Text is relatively expensive because every lit glyph pixel is a separate clipped rectangle fill. The `gpio_get` figure includes the benchmark loop (a modulo and a `volatile` write); the SIO pin read itself is one cycle (6.7 ns). In the full-frame test core 0 works for $19.46 - 18.1 = 1.36$ ms (the clear plus loop overhead) and waits for the rest: the frame time equals the DMA time, confirming that the SPI link, not the CPU, limits the frame rate.

Interrupt load (estimate). The two 1 kHz interrupts each take a few microseconds per call (entry and exit, a handful of pin reads or one PWM update), about 2–4 µs per millisecond in total, i.e. under 0.5 % of core 0.

11.5 Polling and update frequencies

Table 24: Polling frequencies.

Activity	Rate	Mechanism
Button sampling	1 kHz	Timer interrupt, core 0
Sound engine update	1 kHz	Timer interrupt, core 0
Button snapshot (`buttons_poll`)	per frame, ≈51 Hz	Main loop
Joystick ADC (X and Y)	per frame, ≈51 Hz	Main loop
Idle check for auto-sleep	per frame on calm screens	Main loop
Wake check while asleep	1 kHz	Core 0 woken from WFI by the timer interrupt
FPS statistics / serial log	1 Hz	One-second window
Snake movement	6.7–14.3 Hz	150 ms falling to 70 ms
Asteroid spawning	1.1–3.3 Hz	900 ms falling to 300 ms

11.6 Debounce and joystick timing

Debounce: see Table 15 (5 ms, sampled at 1 kHz). Joystick reading: see Table 9.

11.7 Memory usage

Table 25: Memory usage (dual-core build).

Item	Bytes	Share
Flash (4 MB)		
Firmware (code + constants)	65 196 (63.7 KB)	1.6 %
Saved scores sector	4 096	0.1 %
RAM (520 KB)		
Two framebuffers (2 × 153 600)	307 200	57.7 %
Space Shooter state, particles, stars, pop-ups	3 383	0.6 %
Snake body	1 344	0.3 %
Core 1 stack	2 048	0.4 %
USB stack, SDK and other	3 753	0.7 %
Total static RAM	**317 728 (310.3 KB)**	**59.7 %**
Free for heap (main SRAM)	≈208 600 (203.7 KB)	39.2 %

The single-core build needs only one framebuffer and uses about 160 KB of RAM. Dual-core rendering therefore costs 150 KB of RAM in exchange for overlapping drawing with transfer.

11.8 Asset sizes

Because sprites are procedural, the stored graphics and sounds are very small. All of them are declared `const`, so they stay in flash and cost no RAM.

Table 26: Asset sizes (from the firmware symbol table).

Asset	Bytes	Content
5×7 font	195	39 glyphs × 5 columns
Heart icon	6	7×6 bitmap
Sound-effect segments	120	15 segments × 8 B
Sound-effect descriptors	80	10 effects × 8 B
Menu game table	32	2 games × 16 B
Button pin table	20	5 pins
Total stored assets	**453**	
Ship, asteroids, orb, snake, icons	0	drawn procedurally in code
Note frequencies (`notes.h`)	0	compile-time constants

12 Testing and Validation

- **Hardware tests**: each part has a stand-alone program. `display_test` (border, colour bars, moving square, FPS), `buttons_test` (state plus a press counter that must rise by exactly 1 per press, demonstrating the debounce), `joystick_test` (raw and normalised values, for calibration), `sound_test` (scale and sweep), and `benchmark_test` (all timing figures).

- **Flash save logic**: tested on a PC against a simulated flash chip (Section 8).
- **Rendering**: early UI frames were rendered on a PC with stubbed hardware and inspected as images.
- **Code quality**: every source file compiles with `-Wall -Wextra -Wshadow` with zero warnings.
- **On device**: FPS and timing logged over USB; sleep/wake, pause and score persistence across power cycles checked by hand.

13 Limitations and Future Work

- **Not implemented**: enclosure, multi-console play. Multi-console play could use a UART link between two handhelds (three wires: TX, RX, ground) exchanging game state each frame.
- **SPI bandwidth limits FPS** to about 51. Options: redraw only the parts of the screen that changed (dirty rectangles), 16-bit DMA transfers, or the RP2350's programmable I/O (PIO) to drive the panel.
- **Backlight always on**: wiring BL to a GPIO would allow switching it off in sleep and PWM brightness control.
- **Sound**: one voice only (no chords) and a square-wave timbre; a low-pass filter or a PWM-based DAC would soften the tone and allow software volume control.
- **Joystick**: the dead zone causes a small jump in output at its edge; rescaling and auto-calibrating the centre at boot would smooth control.

14 Conclusion

The project delivers a working Pico 2 handheld with a menu and two complete games. It meets the core software requirements with framebuffer rendering, measured FPS, persistent scores, debounced input and dual-core operation, and adds DMA transfers, non-blocking sound, physics, pause and sleep. The main performance result is that the dual-core pipeline lets the display run at the maximum rate the SPI link allows, about 51 FPS, while core 0 stays about 85 % idle. The main remaining work is physical: the enclosure.

A Building and Flashing

```
cmake -S . -B build # board: pico2 (RP2350)
cmake --build build --target handheld # firmware -> build/handheld.uf2

# single-core comparison build
cmake -S . -B build -DHANDHELD_DUAL_CORE=OFF

# flash with picotool, or hold BOOTSEL and copy the .uf2 to the RP2350 drive
picotool load -f build/handheld.uf2 && picotool reboot
```

B Controls

Input	Menu	Space Shooter	Snake
Joystick	Choose game	Move ship	Steer
A	Hold 3 s: reset scores	Fire up	Up
B	Next game	Fire right	Right
C	FPS overlay	Shield	Down
D	Previous game	Fire left; menu on title/pause/game over	Left; menu on pause/game over
Click	Tap: play; hold 2 s: sleep	Start; pause/resume	Pause/resume; play again

C Glossary

Term	Meaning
ADC	Analog-to-digital converter: turns a voltage into a number
CRC32	32-bit cyclic redundancy check: a checksum that detects corrupted data
DMA	Direct memory access: hardware that copies data without the CPU
DMB	Data memory barrier: an instruction that orders memory accesses
DREQ	Data request: a peripheral's signal telling the DMA it is ready for more data
FPS	Frames per second
FPU	Floating-point unit
GRAM	Graphics RAM inside the display controller
ISR	Interrupt service routine: code run when an interrupt fires
LFSR	Linear-feedback shift register: a fast pseudo-random number generator
Mux	Multiplexer: a switch that selects one of several inputs
PLL	Phase-locked loop: multiplies a reference clock to a higher frequency
PWM	Pulse-width modulation: a square wave with adjustable frequency and duty cycle
RGB565	16-bit colour: 5 bits red, 6 green, 5 blue
SEV / WFE	Send event / wait for event: inter-core signalling instructions
SIO	Single-cycle I/O: per-core block for fast GPIO and inter-core signals
SPI	Serial Peripheral Interface: a clocked serial bus
WFI	Wait for interrupt: halts the core until an interrupt arrives
XIP	Execute-in-place: running code directly from flash